

Índices e restrições

Este capítulo descreve alguns aspectos físicos de armazenamento e acesso ao banco de dados. Inclui estrutura de dados auxiliares, tais como os índices, que objetivam melhorar o desempenho de uma consulta pela instrução SQL a um banco de dados com grande volume de informações. Além disso, descreve alguns tipos de restrições de integridade, como restrições de integridade de entidade, de valores nulos, restrições de domínio e restrições de integridade referencial.

1 Índices e SQL

Um índice é uma estrutura de dados utilizada para agilizar a recuperação de registros em resposta a certas condições de pesquisa (Elmasri e Navathe, 2018). A estrutura de um índice é armazenada em um arquivo no disco e contém informações sobre os valores de uma ou mais colunas da tabela e os endereços físicos onde esses registros estão armazenados. Os tipos predominantes de índices são baseados em arquivos ordenados (índices de único nível) e usam estruturas de dados complexas, tais como Árvores B+ (do inglês, B+-trees) ou índices multiníveis, para organizar o índice. É como buscar um assunto em um livro, consultamos o índice para verificar o número da página em que determinado assunto se encontra.

Há vários tipos de índices ordenados (Elmasri e Navathe, 2018), tais como índices primários, índices secundários e índices de clusterização (ou agrupamento). Um índice primário é aquele índice que é construído sobre um campo chave quando este campo é de ordenação do arquivo. Um índice de clusterização é construído sobre um campo de ordenação quando este não é campo chave. Os índices secundários podem ser especificados sobre qualquer campo de um arquivo.

Os índices são considerados parte da linguagem de definição de dados (DDL – *Data Definition Language*) do SQL. Contudo, a partir do SQL-92, teoricamente, não é mais necessário a criação de índices, porque se permite a definição de restrições de integridade. Segundo Ramakrishnam e Gehrke (2011), o SQL não inclui instruções para criação ou eliminação de índices, porém, todo SGBD comercial ainda fornece suporte para alguns tipos de índice.

Para especificar um índice usa-se o comando `CREATE INDEX` e para eliminá-lo usa-se o comando `DROP INDEX`. Caso seja necessário a execução de muitas consultas na tabela `FUNCIONARIO` pelo atributo `sobrenome`, é importante que se crie um índice sobre o campo `sobrenome` na tabela. Para isso, basta executar o comando conforme exemplo 1.

Exemplo 1 – Índice sobre atributo SOBRENOME da tabela FUNCIONARIO

```
CREATE TABLE FUNCIONARIO (  
    NUMAT          INT,  
    NOME           VARCHAR(100) NOT NULL,  
    SOBRENOME      VARCHAR(100) NOT NULL,  
    DTNASC         DATE,  
    ENDERERECO     VARCHAR(50),  
    SALARIO        DECIMAL(10,2),  
    NDEPTO         INT NOT NULL,  
    PRIMARY KEY(NUMAT));  
  
CREATE INDEX IDX_SOBRENOME ON FUNCIONARIO(SOBRENOME);
```

Pode-se definir ordem ascendente (default, ASC) ou descendente (DESC), como se pode perceber no exemplo 2.

Exemplo 2 – Índice composto por atributo SOBRENOME e NOME na tabela FUNCIONARIO

```
CREATE INDEX IDX_NOME ON FUNCIONARIO(SOBRENOME ASC, NOME DESC);
```

Pode-se especificar um índice sobre um atributo chave (único) por meio da palavra-chave UNIQUE, observe o exemplo 3.

Exemplo 3 – Índice único sobre atributo chave na tabela FUNCIONARIO

```
CREATE UNIQUE INDEX IDX_NUMAT ON FUNCIONARIO(NUMAT);
```

Índices de clusterização são índices nos quais a ordem física dos registros de dados de uma tabela é organizada de acordo com um campo não chave – que não tem um valor distinto para cada registro (Elmasri e Navathe, 2018). Para criar um índice de agrupamento, observe o exemplo 4.

Exemplo 4 – Índices de clusterização

```
CREATE INDEX IDX_NDEPTO ON FUNCIONARIO(NDEPTO) CLUSTER;
```

Na definição dos índices sobre as várias tabelas de um banco de dados, toda atualização na tabela base implicará na atualização de todos os índices associados àquela tabela. Além disso, índices sempre requerem armazenamento adicional. Por estas razões, sempre que conveniente, deve-se liberá-los. Para eliminar o índice IDX_NDEPTO basta executar a instrução do exemplo 5.

Exemplo 5 – removendo o índice IDX_NDEPTO

```
DROP INDEX IDX_NDEPTO;
```

2 Restrições padrão

Esta seção apresenta restrições que podem ser especificadas em SQL como parte da criação de uma tabela. As restrições são regras impostas a um banco de dados para garantir a consistência e validade dos dados. As principais são restrições de chave e restrições sobre domínios de atributos e valores nulos, restrições sobre tuplas individuais dentro de uma relação e restrições de integridade referencial (Elmasri e Navathe, 2018).

2.1 Restrições sobre uma relação

O comando CREATE TABLE permite incluir instruções para restrições de integridade (Silberschatz, Korth e Sudarshan, 2020). Além das restrições de chave primária, existem outras restrições que podem ser incluídas no comando CREATE TABLE. As restrições permitidas são:

- **Restrição de valores nulos:** uma restrição NOT NULL pode ser especificada se o atributo não permitir valor nulo.
- **Restrição de valor default:** é possível especificar um valor padrão para um atributo.
- **Restrição UNIQUE:** especifica atributos cujos valores são únicos na tabela, ou seja, não há duas tuplas com os mesmos valores para esses únicos.
- **Cláusula CHECK:** uma cláusula utilizada para limitar valores ou domínio.
 - Por exemplo, suponha que números de departamento sejam restritos a números entre 1 e 20. Então, pode-se alterar a declaração do atributo NUMDEP da tabela DEPARTAMENTO para o seguinte:

```
NUMDEP INT NOT NULL CHECK (NUMDEP > 0 AND NUMDEP < 21)
```

Observe pelo exemplo 6 a aplicação de restrições sobre atributos das tabelas FUNCIONARIO e DEPARTAMENTO.

Exemplo 6 – Tabelas FUNCIONARIO e DEPARTAMENTO

```
CREATE TABLE DEPARTAMENTO (
    NUMDEP NUMBER NOT NULL,
    NOMEDEP VARCHAR(50) NOT NULL,
    DTINI_CHEFE DATE,
    CONSTRAINT PK_DEPTO PRIMARY KEY(NUMDEP),
    CONSTRAINT UK_NOMEDEP UNIQUE(NOMEDEP)
);

CREATE TABLE FUNCIONARIO (
    NUMAT NUMBER NOT NULL,
    NOME VARCHAR(100) NOT NULL,
    SOBRENOME VARCHAR(100) NOT NULL,
    DTNASC DATE,
    ENDEREREÇO VARCHAR(50),
    SEXO CHAR(1) CHECK(SEXO IN ('M', 'F')),
    SALARIO DECIMAL(10,2),
    NSUPER NUMBER,
    NDEPTO NUMBER DEFAULT 1 NOT NULL,
    CONSTRAINT PK_FUNC PRIMARY KEY(NUMAT),
    CONSTRAINT FK_FUNC_SUPERVISOR
        FOREIGN KEY(NSUPER) REFERENCES FUNCIONARIO(NUMAT)
        ON DELETE SET NULL,
    CONSTRAINT FK_FUNC_DEPTO
        FOREIGN KEY(NDEPTO) REFERENCES DEPARTAMENTO(NUMDEP)
        ON DELETE CASCADE
);
```

Fonte: adaptado de Elmasri e Navathe (2018).

O exemplo 6 apresenta o comando CREATE TABLE para criar duas tabelas, a primeira é DEPARTAMENTO e a segunda é FUNCIONARIO. Na tabela DEPARTAMENTO tem a restrição de chave primária sobre o atributo NUMDEP, tem a restrição de valor único sobre o atributo NOMEDEP. Na tabela FUNCIONARIO tem a restrição de chave primária sobre o atributo NUMAT, sobre o atributo sexo tem a restrição de CHECK que verifica que o atributo contém valores 'M' ou 'F' e o atributo NDEPTO que tem a restrição de valor

padrão igual a 1. Se no momento da inserção o atributo NDEPTO não for informado, este assumirá o valor 1. Outras restrições de integridade referencial serão detalhadas em seções posteriores.

2.2 Restrições de domínio

Um usuário pode definir um domínio usando a instrução CREATE DOMAIN, que pode fazer o uso da restrição de CHECK (Elmasri e Navathe, 2018). Observe o exemplo 7 para utilização do comando CREATE DOMAIN.

Exemplo 7 – Restrição sobre domínio

```
CREATE DOMAIN TIPOSEXO AS CHAR(1)
CHECK (VALUE IN ('M', 'F'));
```

Dessa forma, é possível usar o domínio TIPOSEXO para a criação de tabelas garantindo que o atributo SEXO aceite apenas dois valores. Observe o exemplo 8 que mostra o uso em uma tabela.

Exemplo 8 – Tabela FUNCIONARIO com domínio TIPOSEXO

```
CREATE TABLE FUNCIONARIO (
    NUMAT NUMBER NOT NULL,
    NOME VARCHAR(100) NOT NULL,
    SOBRENOME VARCHAR(100) NOT NULL,
    DTNASC DATE,
    ENDEREÇO VARCHAR(50),
    SEXO TIPOSEXO,
    SALARIO DECIMAL(10,2),
    NSUPER NUMBER,
    NDEPTO NUMBER DEFAULT 1 NOT NULL,
    CONSTRAINT PK_FUNC PRIMARY KEY(NUMAT),
    CONSTRAINT FK_FUNC_SUPERVISOR
        FOREIGN KEY(NSUPER) REFERENCES FUNCIONARIO(NUMAT)
        ON DELETE SET NULL,
    CONSTRAINT FK_FUNC_DEPTO
        FOREIGN KEY(NDEPTO) REFERENCES DEPARTAMENTO(NUMDEP)
        ON DELETE CASCADE
);
```

3 Asserções

Em SQL é possível especificar restrições gerais, o que garante que certas condições sejam sempre verdadeiras no banco de dados (Elmasri e Navathe, 2018). Para isso, utiliza-se o comando CREATE ASSERTION. Suponha que na tabela de funcionários nenhum salário deve ser menor do que R\$ 1000,00. Veja como fica a asserção pelo exemplo 9.

Exemplo 9 – Asserção sobre o salário

```
CREATE ASSERTION SALARIO_FUNCIONARIO
CHECK(
  NOT EXISTS (
    SELECT *
    FROM FUNCIONARIO
    WHERE SALARIO < 1000
  )
)
```

O exemplo 9 cria uma asserção para verificar se não existe FUNCIONARIO com SALARIO menor que 1000. Caso alguma inserção ou modificação coloque esse predicado com FALSO, o banco de dados então impede essas alterações.

A asserção verifica continuamente se uma condição específica é satisfeita no banco de dados. No exemplo 9, toda vez que uma tupla é inserida ou modificada na tabela FUNCIONARIO, a asserção verifica se a cláusula CHECK é verdadeira, ou seja, não existe tupla na tabela FUNCIONARIO cujo SALARIO seja menor que 1000. Caso esse predicado seja falso, o banco de dados impede a inserção ou alteração de dados que violem essa regra.

Observação: o banco de dados Oracle não suporta o comando CREATE ASSERTION.

4 Restrição de integridade referencial

A integridade referencial é especificada por meio da cláusula FOREIGN KEY (Elmasri e Navathe, 2018), como se observa no exemplo 6. Essa restrição assegura que, se uma chave estrangeira é usada, ela deve referir-se a uma chave primária existente na tabela relacionada (Ramakrishnan e Gehrke, 2011).

Se uma tupla for inserida ou atualizada com um valor na chave estrangeira que não existe na chave primária correspondente, o banco de dados impedirá essa operação para manter a consistência dos dados.

Perceba que no exemplo 6, o atributo NDEPTO da tabela FUNCIONARIO é uma chave estrangeira que faz referência ao atributo NUMDEP da tabela DEPARTAMENTO. Dessa forma, um funcionário sempre estará associado a um departamento válido.

Ainda no exemplo 6, é possível observar que a restrição de integridade referencial ocorre no atributo NSUPER, uma chave estrangeira que faz referência ao atributo NUMAT da própria tabela de FUNCIONARIO. Isso caracteriza um relacionamento recursivo ou autorrelacionamento.

5 Tratamento de violação de restrições de integridade referencial

O padrão SQL fornece algumas ações a serem tomadas quando ocorrem violações de integridade. A ação ocorre quando uma tupla da tabela referenciada é excluída (ON DELETE) ou atualizada (ON UPDATE). Isso é feito por meio das opções:

- **SET NULL:** se uma tupla na tabela referenciada for excluída ou atualizada, a chave estrangeira da tupla correspondente na tabela será definida como NULL.

- **CASCADE:** se uma tupla na tabela referenciada for excluída ou atualizada, as tuplas correspondentes na tabela também serão excluídas ou atualizadas.
- **SET DEFAULT:** define um valor padrão para a chave estrangeira, caso a tupla correspondente na tabela referenciada seja excluída.
- **RESTRICT:** é a ação padrão que o SQL toma para uma violação de integridade, rejeitando qualquer operação de atualização que possa causar uma violação.

Observe que no exemplo 6, o atributo NSUPER é chave estrangeira e faz referência ao atributo NUMAT na própria tabela FUNCIONARIO. Caso o supervisor seja excluído, todos os subordinados terão sua chave estrangeira atualizada para NULL.

Se o atributo NDEPTO é chave estrangeira e faz referência ao atributo NUMDEP na tabela DEPARTAMENTO, caso o departamento correspondente a NDEPTO seja excluído, então todos os funcionários que trabalham naquele departamento automaticamente serão removidos, devido à cláusula ON DELETE CASCADE.

Considerações finais

Este capítulo apresentou aspectos físicos do banco de dados, iniciando pelo conceito de índices. O texto descreveu os tipos de índices que podem ser primários, secundários e índices de clusterização (ou agrupamento).

Na sequência, vimos algumas restrições (padrões que podem ser especificados na criação de uma tabela) o que inclui restrições de valores nulos, restrições de valores padrões e restrições de valores únicos. Após isso, apresentou-se o comando CREATE DOMAIN para criar um domínio que pode ser utilizado na criação da tabela e a restrição CHECK aplicada a um domínio.

Por fim, vimos as restrições de integridade referencial com a cláusula FOREIGN KEY para o estabelecimento de uma chave estrangeira que faz referência a uma outra tabela no banco de dados, e ações para tratar as violações de restrição de integridade referencial.

Referências

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018.

RAMAKRISHNAN, Raghu; GEHRKE, Johannes. **Sistema de gerenciamento de banco de dados**. 3. ed. Porto Alegre: AMGH, 2011.

SILBERSCHATZ, Abraham; KORTH, Henry F; SUDARSHAN, S. **Sistema de Banco de Dados**. 7. ed. Rio de Janeiro: GEN LTC, 2020.